



Mini-Project 5

AI-Powered Airline Customer Support System

Deployment on Hugging Face Spaces

In this mini-project, you will containerize the previously developed **AI-Powered Airline Customer Support System** using Docker and deploy it on **Hugging Face Spaces**. The activity focuses on packaging the application, configuring the deployment environment, and making the end-to-end solution accessible through a cloud-hosted interface.

[Mini-project Session - 28th June 2026]

Step 1: Verify the Complete Application Code

1.1 Ensure that you have the complete backend and frontend code of the previously developed **AI-Powered Airline Customer Support System**.

1.2 Verify that the project contains all required files, including:

- FastAPI backend files
- Streamlit frontend files
- RAG and agent-related components
- Database connection code
- Guardrail components
- requirements.txt
- Environment-variable configuration
- Supporting datasets and knowledge-base documents

1.3 Open the project in [GitHub Codespaces](#) and review the folder structure and code organization.

Step 2: Test the Application Before Containerization -- (2 Points)

2.1 Install all required Python dependencies

2.2 Configure the required environment variables and API keys

2.3 Start the FastAPI backend in one terminal



2.4 Start the Streamlit frontend in another terminal

2.5 Open the forwarded ports in GitHub Codespaces and verify that:

- The frontend loads successfully
 - The frontend can communicate with the backend
 - User queries receive valid responses
 - Database, RAG, and agent components work as expected
-

Step 3: Dockerize the Application in GitHub Codespaces -- (2 Points)

3.1 Create a **Dockerfile** in the root directory of the project

3.2 Add instructions to:

- Select an appropriate Python base image
- Copy the application files into the container
- Install the required dependencies
- Expose the application port
- Start the backend and frontend services

3.3 Create a **.dockerignore** file to exclude unnecessary files such as: `.git`, `__pycache__`, `.env`, `.venv`, `*.pyc`

3.4 Build the Docker image

3.5 Verify that the image was created successfully

Step 4: Test the Docker Image by Running a Container -- (2 Points)

4.1 Run a container using the newly created Docker image

4.2 Pass the required environment variables using an environment file, when applicable

4.3 Check whether the container is running

4.4 Review the container logs

4.5 Open the application through the forwarded port and verify its functionality



Step 5: Push the Docker Image to Docker Hub -- (1 Point)

- 5.1 Create a Personal Access Token in your [Docker Hub](#) account, or use an existing one
 - 5.2 Log in to Docker Hub from GitHub Codespaces
 - 5.3 Tag the local Docker image using your Docker Hub username
 - 5.4 Push the tagged image to Docker Hub
 - 5.5 Open Docker Hub and confirm that the image is available in the repository
-

Step 6: Deploy the Application on Hugging Face Spaces -- (2 Points)

- 6.1 Log in to [Hugging Face](#) and create a new Space
 - 6.2 Select **Docker** as the Space SDK.
 - 6.3 Configure the Space name, visibility, and hardware settings.
 - 6.4 Add sensitive values such as API keys, database credentials (Supabase, Pinecone, etc), and tokens under the Space's **Secrets and Variables** settings.
 - 6.5 Edit the **README.md** and specify the app port
 - 6.6 Create the **Dockerfile** and mention your docker image as base image
 - 6.7 Review the build logs and resolve any dependency, port, or configuration errors
 - 6.8 Wait for the Space status to change to **Running**
-

Step 7: Access and Test the Deployed Application -- (1 Point)

- 7.1 Open the public URL generated for the Hugging Face Space.
- 7.2 Verify that the application interface loads without errors.



7.3 Test representative airline customer-support queries related to:

- Flight status
- Baggage policies
- Check-in
- Boarding passes
- Identity documents
- Other airline FAQs

7.4 Confirm that the deployed application can successfully access the required APIs, databases, vector stores, and environment variables

7.5 Review the Hugging Face Space logs for any runtime errors